MATH 345 Skeleton Notes

Unit 1: Vectors, matrices, and linear maps

Section 1.4: Matrix multiplication and map composition

Matrix product by columns

Definition: Matrix-matrix product Let A be an $m \times n$ matrix, let B be an $n \times k$ matrix, and write

$$B = \begin{bmatrix} \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_k \end{bmatrix}$$

where $\mathbf{b}_j$ is column j of B. The $\boxed{\text{product}}$ AB is the $m \times k$ matrix

$$AB = \begin{bmatrix} A\mathbf{b}_1 & A\mathbf{b}_2 & \cdots & A\mathbf{b}_k \end{bmatrix}.$$

Activity: Computing a matrix product using columns

Let

$$A = \begin{bmatrix} 1 & -3 & 5 \\ 3 & 1 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 5 & 1 \\ -2 & -6 \\ -1 & 0 \end{bmatrix}.$$

Compute AB.

Entry formula by row-column dot products

Theorem If A is $m \times n$ and B is $n \times k$, then the (i, j)-entry of AB is the dot product of row i of A with column j of B:

$$(AB)_{ij} = \sum_{\ell=1}^{n} a_{i\ell} b_{\ell j}.$$

Activity: Computing a matrix entry

If

$$A = \begin{bmatrix} -3 & -4 & 1 \\ 2 & 4 & 0 \\ 1 & -4 & -5 \end{bmatrix}, \qquad B = \begin{bmatrix} -1 & 2 & -4 \\ -4 & -3 & -1 \\ 4 & 3 & 1 \end{bmatrix},$$

what is the $(2, 3)$-entry of AB?

Theorem: Properties of matrix multiplication Assume that A, B, C are matrices of appropriate sizes and r is a scalar. Then

1. $A(BC) = (AB)C$.

2. $(A + B)C = AC + BC$.

3. $C(A + B) = CA + CB$.

4. $A(rB) = r(AB) = (rA)B$.

5. $(AB)^T = B^T A^T$.

Matrix multiplication as composition

If $T_A(\mathbf{x}) = A\mathbf{x}$ and $T_B(\mathbf{x}) = B\mathbf{x}$, then applying B first and then A gives

$$T_A(T_B(\mathbf{x})) = A(B\mathbf{x}) = (AB)\mathbf{x}.$$

Matrix multiplication represents composition of matrix maps.

Activity: Composition shape check

Suppose A is 4×3 and B is 3×2. What is the shape of AB, and what is the domain and codomain of the composed map $\mathbf{x} \mapsto A(B\mathbf{x})$?

Activity: Applying one matrix to many points

Put the vertices of a shape in the columns of a matrix X. Then $Y = AX$ contains the transformed vertices.

```
import numpy as np

np.set_printoptions(precision=3, suppress=True)

# Columns are vertices of the unit square.
# The final column repeats the first vertex to close the shape.
X = np.array([
    [0, 1, 1, 0, 0],
    [0, 0, 1, 1, 0],
], dtype=float)

A = np.array([
    [1, 1],
    [0, 1],
], dtype=float)

Y = A @ X
Y
```

Output:

```
array([[0., 1., 2., 1., 0.],
       [0., 0., 1., 1., 0.]])
```

1. What are the columns of `X`?
2. What are the columns of `Y`?
3. Which transformation from the gallery is this?
4. Why does the bottom edge stay fixed?
5. Why does the top edge shift right?

Warning In this geometric example, the columns of X are points, so $Y = AX$ applies the same matrix to every point. In many data tables, observations are stored as rows. For example, a document matrix may have one document per row. Always check what rows and columns represent before interpreting a product.

Note: Later square-grid visualizations The matrix computation $Y = AX$ applies one matrix to every column of X. For a 2×2 matrix, the images of the coordinate directions determine the whole grid.
Later, in Corners are not enough, the same columns-of-points convention is used for rules that are not matrix maps. Then four corners may not tell the whole story. A grid of input points gives more information.

A missing rule: AB need not equal BA

Multiplication of real numbers is commutative: $ab = ba$. Matrix multiplication is not commutative in general. For two square matrices A and B of the same size, we say that A and B commute when

$$AB = BA.$$

Some pairs commute and some do not.

Activity: Commute or not?

For each pair below:

1. compute both products;
2. determine whether $AB = BA$;

Pair A: two distinct diagonal transformations. Use the horizontal stretch and reflection across the y-axis:

$$S = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \qquad F = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Pair B: a shared axis label is not enough. Use the same horizontal stretch and the horizontal shear:

$$S = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \qquad H = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Pair C: reflection across the line $y = x$. Use the same horizontal stretch and

$$M = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Tags. `[U1-L06 | P+R | Core]`

Activity: Two horizontal shears

Let

$$H = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad G = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}.$$

The matrix G is a stronger horizontal shear; it is also the result of applying H twice. Compute HG and GH. Do the matrices commute?
Tags. `[U1-LO6 | P+R | Core]`

Note Some structural patterns settle the question quickly. Two diagonal matrices of the same size commute, and every matrix commutes with itself. The two horizontal shears above also commute. Different geometric moves often do not commute, but this is a warning pattern, not a rule. When no structural pattern settles the question, compute AB and BA and compare.

Many token queries and keys

Example: Many token queries at once Suppose a sequence has L tokens. Store the token vectors as rows of a matrix

$$X = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_L^T \end{bmatrix} \in \mathbb{R}^{L \times d}.$$

A transformer layer forms three views of the same token vectors:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

The matrices W_Q, W_K, W_V are weight matrices. Their entries are adjusted during training. For now, treat them as ordinary matrices that assign each token three roles.

For token i, the vector $\mathbf{q}_i$ is its query: what token i uses to decide which tokens matter for its update.

For token j, the vector $\mathbf{k}_j$ is its key: what token j uses to be compared against a query.

For token j, the vector $\mathbf{v}_j$ is its value: what token j can contribute after weights are chosen.

Write

$$Q = \begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_L^T \end{bmatrix}, \qquad K = \begin{bmatrix} \mathbf{k}_1^T \\ \mathbf{k}_2^T \\ \vdots \\ \mathbf{k}_L^T \end{bmatrix}.$$

Then

$$K^T = \begin{bmatrix} \mathbf{k}_1 & \mathbf{k}_2 & \cdots & \mathbf{k}_L \end{bmatrix}.$$

So

$$QK^T = \begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_L^T \end{bmatrix} \begin{bmatrix} \mathbf{k}_1 & \mathbf{k}_2 & \cdots & \mathbf{k}_L \end{bmatrix} = \begin{bmatrix} \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \cdots & \mathbf{q}_1 \cdot \mathbf{k}_L \\ \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \cdots & \mathbf{q}_2 \cdot \mathbf{k}_L \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{q}_L \cdot \mathbf{k}_1 & \mathbf{q}_L \cdot \mathbf{k}_2 & \cdots & \mathbf{q}_L \cdot \mathbf{k}_L \end{bmatrix}.$$

Thus the score matrix

$$S = QK^T$$

has entries

$$S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j.$$

Row i of S contains the scores used when updating token i.

Activity: Many-token shape and entry check

Suppose

$$X \in \mathbb{R}^{5\times 4}$$

and

$$W_Q, W_K, W_V \in \mathbb{R}^{4\times 3}.$$

Let

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad S = QK^T.$$

1. What are the shapes of Q, K, and V?
2. What is the shape of K^T?
3. What is the shape of $S = QK^T$?
4. Suppose

   $$\mathbf{q}_2 = \begin{bmatrix} q_{21} \\ q_{22} \\ q_{23} \end{bmatrix}, \qquad \mathbf{k}_4 = \begin{bmatrix} k_{41} \\ k_{42} \\ k_{43} \end{bmatrix}.$$

   Expand the entry S_{24}.
5. What does S_{24} compare?
6. What does row 2 of S contain?

For this section, an attention calculation has three steps:

$$\text{dot-product scores} \quad \longrightarrow \quad \text{weights} \quad \longrightarrow \quad \text{weighted averages.}$$

The score S_{ij} compares token i's query with token j's key. A larger score means token j is more relevant when updating token i.
The scores are not yet the output. For each token i, a weighting rule converts row i of S into numbers

$$\alpha_{i1}, \alpha_{i2}, \ldots, \alpha_{iL}$$

that are nonnegative and add to 1. These are the attention weights for token i. They say how much token i uses each value vector.
Let

$$A_{\text{att}} = [\alpha_{ij}]$$

be the attention-weight matrix. If the value vectors are stored as rows,

$$V = \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_L^T \end{bmatrix},$$

then the attention output is

$$H = A_{\text{att}} V.$$

Row i of H is

$$\mathbf{h}_i^T = \alpha_{i1}\mathbf{v}_1^T + \alpha_{i2}\mathbf{v}_2^T + \cdots + \alpha_{iL}\mathbf{v}_L^T.$$

Equivalently,

$$\mathbf{h}_i = \alpha_{i1}\mathbf{v}_1 + \alpha_{i2}\mathbf{v}_2 + \cdots + \alpha_{iL}\mathbf{v}_L.$$

So each updated token vector is a weighted average of value vectors.

Activity: Many-token attention output

Suppose

$$X \in \mathbb{R}^{4\times 3}, \qquad W_Q, W_K, W_V \in \mathbb{R}^{3\times 2}.$$

Define

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad S = QK^T.$$

A weighting step converts S into an attention-weight matrix

$$A_{\text{att}} \in \mathbb{R}^{4\times 4}.$$

The attention output is

$$H = A_{\text{att}}V.$$

1. What are the shapes of Q, K, and V?
2. What is the shape of $S = QK^T$?
3. What does S_{ij} compare?
4. What does row i of A_{att} tell you?
5. What is the shape of H?
6. Why is each row of H a weighted average of value vectors?
7. Now suppose row 2 of A_{att} is

   $$\alpha_2^T = \begin{bmatrix} 0 & 1/2 & 1/4 & 1/4 \end{bmatrix}$$

   and

   $$V = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 0 & 4 \\ 4 & 4 \end{bmatrix}.$$

Warning In real transformer attention, the weighting rule usually uses scaled and masked scores before forming weights. For Unit 1, the important facts are: rows of A_{att} are weights, the weights add to 1, and $A_{\text{att}}V$ forms weighted averages of the rows of V.

Activity: The same shape check in code

The following code performs a many-token shape check similar to the previous activity.

```
import numpy as np

X_tokens = np.ones((5, 4))
WQ = np.ones((4, 3))
WK = np.ones((4, 3))
WV = np.ones((4, 3))

Q = X_tokens @ WQ
K_tokens = X_tokens @ WK
V_tokens = X_tokens @ WV
scores_many = Q @ K_tokens.T

Q.shape, K_tokens.shape, V_tokens.shape, scores_many.shape
```

Output:

```
((5, 3), (5, 3), (5, 3), (5, 5))
```

1. Which line creates all query-key scores at once?
2. Why is `scores_many.shape` equal to `(5, 5)`?